

Apart × Atlas — Secure Program Synthesis Fellowship Proposal

Target output

Translation-validation tool for non-interference preservation across MLIR lowering, evaluated on HEIR's **secret** dialect. Target: CSF 2026 (deadline ~Feb 2026) or TACAS / FMCAD as backup.

Venue rationale. Submission cycles relevant to the fellowship calendar: CSF 2026 (paper deadlines historically Feb / May / Aug — three rounds per year, IEEE CSF), TACAS 2027 (ETAPS, typically October submission), ICSE 2027 (typically March submission, longer review). CSF is the sharpest fit (it's the venue Barthe-style non-interference work has landed in for fifteen years); TACAS catches the formal-methods audience if the SMT/Lean engineering is the stronger story by mid-fellowship; ICSE is a fallback if the artifact and empirical evaluation crystallize faster than the theory.

Summary

Model weights are confidential and valuable, but the compilation pipeline that turns a model into an executable is an attack surface that current ML compilers do not defend. [Chen et al. \(IEEE S&P 2026\)](#) established this for *integrity*: the compiler can inject backdoors. We address the parallel **confidentiality** problem: lowering passes in MLIR can introduce information channels (data-dependent control flow, memory aliasing, dynamic shapes) that leak protected weights even when the source-level program is non-interferent. Existing MLIR translation-validation tools ([mlir-tv](#), [Regehr PLDI'25](#)) check equivalence, not non-interference. We extend them to check **non-interference preservation** under lowering, evaluated on [HEIR's secret](#) dialect. Deliverables: (i) a self-composition-based SMT encoding on top of mlir-tv, (ii) evaluation against ~10 HEIR-relevant lowerings, (iii) a syntactically-checkable "data-independent lowering" subset proved sound. Expected outcome: at least one leak-class identified in upstream lowerings.

High-level background

Four foundational lines of work this project sits between:

1. **The compiler as an ML attack surface.** Chen, Peng, He, Yang, Ray, [Your Compiler is Backdooring Your Model: Understanding and Exploiting Compilation Inconsistency Vulnerabilities in Deep Learning Compilers](#) (IEEE S&P 2026). This is the key motivating result. Chen et al. show that production DL compilers can be coerced into producing backdoored binaries from benign source models — 100% attack success on triggered inputs, zero detection by current defenses, and 31 of the top 100 HuggingFace models already contain *natural* compilation- induced triggers exploitable without modifying source. **This establishes that the MLIR-level compilation pipeline is a real attack surface for ML.** Chen et al. attack the *integrity* property: post-compilation semantics diverge from source. This project addresses the parallel *confidentiality* property: lowering passes can introduce information channels that leak protected weights. The threat surface is the same — the compiler — and we expect the defensive infrastructure to share most of its machinery, but the security property and proof obligations differ.
2. **Information flow control via type systems.** Volpano, Smith, and Irvine, [A sound type system for secure flow analysis](#) (JCS 1996). Barthe, D'Argenio, Rezk, [Secure information flow by self-composition](#) (CSFW 2004; journal version MSCS). These give the relational semantics for non-interference and the self-composition reduction that lets a unary verifier discharge a relational obligation. Our SMT encoding is in this tradition.
3. **Translation validation for compiler IRs.** Pnueli, Siegel, Singerman, *Translation validation* (TACAS 1998). Lopes, Lee, Hur, Liu, Regehr, [Alive2: Bounded Translation Validation for LLVM](#) (PLDI 2021, Distinguished Paper) — bounded SMT-based equivalence checking for LLVM IR peephole optimizations, which found 47 bugs in production LLVM. Regehr et al., [First-Class Verification Dialects for MLIR](#) (PLDI 2025) — ports this discipline to MLIR and validates a canonicalization pass and dataflow analysis transfer functions. Our work generalizes the Alive2 / Regehr discipline from semantic equivalence (a unary post-condition) to non-interference (a relational post-condition).
4. **Confidentiality in ML systems.** The weight-extraction threat model is established in the ML security literature: model extraction attacks via query access ([Tramèr et al., USENIX 2016](#)), weight stealing via side channels ([Batina et al., CSI NN, USENIX 2019](#)), and the broader literature on confidentiality of models as IP and as the carrier of training-data privacy. The compilation-induced leak vector — that lowering itself can introduce channels that were not present in the source program — is the under-studied piece. Existing IFC work for compilers ([Sison & Murray, Verifying that a compiler preserves concurrent value-dependent information-flow security](#), 2019, on non-interference preservation in an Isabelle/HOL-verified compiler; [Almeida et al. on Jasmin's constant-time preservation](#)) lives at LLVM or assembly-DSL altitude and does not reach modern ML compiler IRs.

How our work relates to Chen et al. specifically. This is a parallel, not a direct response. Chen et al. demonstrate that compilation can break a desired *integrity* invariant (functional equivalence) and that the security community has noticed. We argue — and aim to demonstrate empirically — that compilation can equally break a desired *confidentiality* invariant (non-interference between secret weights and public outputs), via similar mechanisms: lowering passes that introduce data-dependent control flow on a secret tensor, canonicalizations that drop encoding metadata carrying security labels, bufferization that exposes weights via memory-aliasing patterns visible to a co-tenant. If Chen et al. is the first paper to treat the DL compiler as an integrity attack surface, this is the parallel first paper for confidentiality. (Hedge: the "first paper" framing depends on the systematic literature review the fellow will do in M1; we adjust the claim downward if priors exist.)

Comparison to similar goals. [SecWasm \(Bastys, Algehed, Sjösten, Sabelfeld, SAS 2022\)](#) addresses IFC for a different IR (WebAssembly), with no lowering- preservation story. [FaCT \(Cauligi et al., PLDI 2019\)](#) addresses constant-time programming at the source-language DSL level. [Jasmin \(Almeida et al., CCS 2017\)](#) provides verified cryptographic-grade constant-time-preserving compilation but for a small assembly-like DSL, not for ML compilation, and operates on confidentiality-via-timing rather than information-flow at the IR level. None of these target the MLIR ecosystem, and none address the compilation-introduced-confidentiality-channel problem at the linalg-on-tensors altitude where ML programs live.

Detailed technical background

The two works the project *directly* builds on:

W1. Regehr et al., [First-Class Verification Dialects for MLIR](#) (PLDI 2025)

Core concept. Rather than writing a one-off equivalence checker for each MLIR dialect, define a small set of *semantic dialects* (e.g., an SMT-backed assertion dialect, a dataflow-fact dialect) and require domain dialects to lower into them. A standard SMT solver then handles any verification expressible in the semantic dialect. The framework is generic over the property being verified — equivalence is just one choice of post-condition. Validated on five upstream dialects; surfaced five miscompilation bugs in production MLIR; verified a canonicalization pass and several dataflow transfer functions end-to-end.

Specific results we extend. The paper proves that lowering pass P preserves the semantics of dialect D iff the SMT formula encoding " $D\text{-semantics}(\text{input}) = D\text{-semantics}(P(\text{input}))$ " is unsatisfiable. We need the *relational* generalization: "for all pairs of inputs agreeing on the low-security projection, P preserves agreement on the low-security projection of the output."

Their semantic-dialect framework is the right place to express the additional relational fact, but the paper does not work it out.

Limitations / gaps we address. (i) Property is fixed to equivalence; no support for relational properties. (ii) No notion of security labels on values; the framework treats all values as observable. (iii) Self-composition (the standard route to encoding relational properties as unary ones) is not discussed and would have to be implemented from scratch. (iv) HEIR's `secret` dialect is not among the five evaluated dialects.

W2. [aqjune/mlir-tv](https://github.com/aqjune/mlir-tv) (Lee, Lopes, Regehr, ongoing)

Core concept. Bounded SMT-based translation validation for MLIR programs, modeled directly on Alive2's approach for LLVM IR. Encodes MLIR tensor and `linalg` ops as SMT formulas over arrays / bitvectors, encodes the lowering pass's output the same way, and asks the solver whether the two are equivalent up to refinement (undefined-behavior weakening). Designed for the dialects most used in ML compilation: `tensor`, `linalg`, `arith`, `memref`, parts of `affine`.

Specific results we extend. The tool already encodes `linalg.generic` (the regional op underlying most ML kernels) into SMT. Encoding HEIR's `secret.generic` requires almost the same machinery — `secret.generic` is structurally analogous, with the region carrying a cleartext block-argument view of the secret operand. We extend the encoding to (a) accept the `secret` dialect's ops, (b) accept the self-composed pair-program representation, and (c) attach a low-security projection to each value derived from the source program's `aisec.protected` annotations.

Limitations / gaps we address. (i) `secret` dialect not supported. (ii) No relational queries; tool emits unary equivalence obligations only. (iii) Per-value security labels are not part of the SMT encoding at all — every value is treated as fully observable. (iv) Soundness of the SMT encoding has not been mechanized; we don't propose to mechanize the tool itself but we do propose to mechanize the dialect semantics (see OP-6 / stretch).

Why these are the right foundations rather than starting from scratch. Building an MLIR translation validator de novo would consume the entire fellowship before the relational extension could even be attempted. `mlir-tv` + the Regehr framework cover (a) parser and SMT glue for the relevant dialects, (b) refinement-checking discipline, (c) a validated SMT encoding of `linalg.generic`. The fellow's contribution sits on top.

Proposed methodology

Hypothesis

H. A self-composition encoding extended with per-value security labels, layered on top of mlir-tv's existing SMT encoding for the `linalg` and `tensor` dialects, suffices to detect non-interference- violating lowerings on a corpus of HEIR-relevant MLIR programs. We further conjecture that a syntactically-checkable subset of lowering passes — those whose rewrite rules are *data-independent* in a precise sense (no rewrite condition depends on a secret-derived value) — preserves non-interference by construction, and we can characterize this subset formally.

Novel techniques

1. **Relational SMT encoding for MLIR with security labels.** Self- composition transformation at the MLIR level (operates on the IR, not on the SMT formula) followed by lifting through mlir-tv's existing encoder. The key engineering bit is preserving the `aisec.protected` / `aisec.declassify` annotations through the self-composition transform so the SMT obligation knows which values are public-observable.
2. **Data-independent lowering subset.** A static characterization of MLIR `PatternRewriter` rules whose match predicates and rewrites do not branch on (or compute over) secret-labeled values. Programs in this subset are non-interference-preserving by construction; a linter pass can certify this without any SMT.
3. **Counterexample lifting.** When the SMT solver returns SAT (non-interference is *not* preserved), lift the counterexample back to a pair of MLIR inputs that produce different public outputs after lowering but agreed on public outputs before. This is the practical artifact: it turns an unsatisfiability failure into a concrete attack witness.

Experimental design

Corpus. ~10 lowering passes from upstream MLIR + HEIR, focused on those that interact with `secret`-typed or `tensor`-typed values:

- `linalg-on-tensors` → `linalg-on-buffers` (bufferization, known-tricky)
- `linalg` → `loops` (the canonical structured-to-unstructured lowering)
- `secret` → `cleartext` rewrite passes inside HEIR
- `secret.generic` outlining / inlining transforms
- A small number of canonicalization patterns that touch `tensor` insert/extract ops (known to drop encodings; see OP-1 in `open-problems.agents.md`)

For each pass, run the relational validator on a hand-crafted suite of ~20 inputs (positive: non-interferent both pre- and post-; negative: manually-introduced leaks) plus a fuzz-generated set if time permits.

Baselines. Two:

1. **Unary equivalence (mlir-tv vanilla).** Does mlir-tv-as-shipped already detect these leaks because they manifest as semantic divergence? Empirical question, but our prior is no: the canonical non-interference leak (a public output depending on a secret input without declassification) is *not* a semantic-equivalence bug, because the pre- and post-lowered programs compute the same function of the secret input. Demonstrating this difference empirically is part of the contribution.
2. **Pre-lowering verifier only (this repo's MVP).** Our prototype verifier checks non-interference at one IR level. It does not address whether lowering preserves the property. So: programs verified pre-lowering, when lowered by a hostile or buggy pass, may emit non-interferent-looking output that is no longer non-interferent. Quantifying this gap is the threat-model motivation for the relational TV story.

Key assumptions

A1. **mlir-tv's SMT encoding of `linalg.generic` and `tensor ops` is sound enough to extend.** If the underlying encoder has soundness bugs, our relational extension inherits them. We will not re-validate mlir-tv; we will treat it as a trusted base and flag any divergence the extension surfaces.

A2. **HEIR's `secret` dialect is stable enough to target.** HEIR is moving fast (2024–2026). We pin a HEIR commit at fellowship start and rebase only if a hard reason emerges. Risk: the dialect's surface syntax shifts mid-fellowship; mitigation: maintain a minimal `secret`-dialect mirror in our repo and target that if upstream churn becomes blocking.

A3. **Self-composition is the right relational encoding for MLIR.** The alternative is a product-program construction that doesn't duplicate operations. Self-composition is simpler and is standard for type-system-style IFC. We do not claim it is optimal; we claim it is sufficient for an evaluation against ~10 lowerings.

A4. **A meaningful fraction of HEIR-relevant lowerings are *not* in the obviously-safe subset.** If almost every relevant lowering turns out to be syntactically data-independent, the SMT machinery has nothing interesting to say and the paper collapses to the characterization theorem alone. We have weak preliminary evidence against this collapse (bufferization legitimately introduces data-dependent control flow), but it's a real falsifier.

How we confirm or deny the hypothesis

Confirming evidence. (i) The relational validator detects at least one non-interference violation in a real upstream MLIR lowering that mlir-tv vanilla does not detect. (ii) The data-independent subset is non-trivial (at least N% of HEIR lowerings fall outside it). (iii) The mechanized soundness theorem for the data-independent subset goes through in Lean 4 with no **sorry**s. Any one of these constitutes a positive result; all three is a strong paper.

Disconfirming evidence. (i) Every realistic HEIR lowering turns out to be syntactically data-independent → the SMT story is unmotivated, write up the characterization theorem alone, target a shorter venue. (ii) The SMT encoding is too expensive to terminate on realistic programs → fall back to bounded depth, write up the scaling limits honestly. (iii) The self-composition encoding has soundness issues we can't resolve → pivot to the Lean mechanization as the primary contribution (this is what OP-6 is for in reserve).

The fellowship is long enough to detect any of these failure modes in time to pivot, provided we are honest about the disconfirming signals as they arrive.

Preliminary experiments

The supporting evidence here is genuinely thin and I want to flag that explicitly rather than dress it up. What exists today in [the repo](#):

1. **A working-on-paper MVP verifier** for non-interference modulo declassification, in C++ on top of HEIR's **secret** dialect. ~200 lines in `lib/Transforms/VerifyNonInterference.cpp`. The verifier is not currently buildable on the maintainer's development environment due to toolchain incompatibilities; this is the meta-problem captured as OP-7 in `docs/exobrain/open-problems.agents.md` and we expect the fellow to inherit a buildable artifact (we will fix the build before fellowship start as a precondition).
2. **Hand-traced execution of nine test cases** (`docs/exobrain/traces.agents.md`), covering positive (`@no_protected`, `@sanctioned_declassify`, `@mobilenet_block_sanctioned`, `@internal_multi_use_ok`) and negative (`@direct_leak`, `@leak_after_generic`, `@mobilenet_block_leak`, `@fanout_via_two_reveals`, `@fanout_via_two_generics`) cases. All nine traces produce the expected diagnostic. **Honest caveat:** hand traces against self-authored code are consistency evidence, not correctness evidence. The fellow will be expected to convert these into real unit tests as the first week's work.

3. **A literature sweep** ([docs/exobrain/findings.agents.md](#)) that found no published MLIR IFC dialect, no published translation validation for non-interference in MLIR, and confirmed the Chen et al. attack as a published high-profile motivation. The sweep was a few hours of web search, not a systematic review. The "first to combine TV with non-interference in MLIR" claim is defensible at *that* search depth and should be hedged in the eventual paper to that depth.
4. **A research trail** ([docs/exobrain/](#)) documenting the design decisions, dead-ends, and reframings that produced the open- problems doc. Worth showing a fellow as evidence that the project has been honestly red-teamed against itself.

What we **do not** have: any quantitative result, any benchmark, any actual SMT-solving experiment, any Lean code. The fellowship is where those come from.

Potential applications & practical uses

Model weights are confidential — they encode training data, are expensive to produce, and increasingly drive commercial value. [Chen et al. \(S&P 2026\)](#) showed the compilation pipeline is an attack surface for ML *integrity*; the same surface is a candidate attack vector for *confidentiality*, with no current defense. A working non-interference-preservation tool would (i) let [HEIR](#) and similar projects ship lowering passes with a checked guarantee they do not introduce weight-leak channels, (ii) give MLIR pass authors a CI-grade regression catch for security properties, on par with existing equivalence-checking, (iii) supply the formal-methods community with a working example of relational verification at modern compiler-IR altitude, and (iv) seed a longer agenda in confidentiality-preserving ML compilation: quantitative info flow, bandwidth-bounded egress certificates, compositional FHE-scheme proofs (OP-4, OP-5 in [docs/exobrain/open-problems.agents.md](#)). Practical adoption is gated on HEIR's trajectory, which we don't control.

Risks, uncertainties, monitoring, & mitigation

I'll red-team in the spirit of "what is the best reason not to fund this."

R1. The novelty claim is weaker than it looks. A deeper literature search — properly through DBLP, the CSF and CCS proceedings of the last five years, plus arXiv — may turn up an existing MLIR-or-adjacent IFC preservation result we missed. **Monitoring:** the fellow does a full systematic literature review in week 1–2, *before* writing any code, and we update the framing if the gap turns out to be smaller. The project survives even if "first" can't be claimed — translation

validation for an MLIR-specific dialect (HEIR's `secret`) is still a contribution at the workshop tier.

R2. The SMT encoding doesn't scale. Self-composition doubles the program size and self-composed non-interference obligations are known hard. Realistic ML kernels (a `linalg.matmul` over `tensor<256x768>`) may not terminate within a reasonable solver budget. **Mitigation:** work small. Scope evaluation programs to the smallest interesting fragment of a real model (e.g., a single MobileNet block, not a full network). Use bounded model checking with explicit depth limits and report scaling limits honestly. Don't pretend to verify a full transformer.

R3. HEIR moves under us. Already happened during prototype development; not hypothetical. **Mitigation:** pin commit at start. Maintain a minimal local mirror of the `secret` dialect ops we need. Don't rebase mid-fellowship unless there's a hard reason.

R4. The build problem is real. This repo has not been successfully built on Arch + GCC 15.2 due to LLVM ABI churn. We will not ask the fellow to inherit an unbuildable artifact.

R5. Capability hazard (dual use). This is more delicate than for Chen et al. and I want to be honest about it. Chen et al. could appeal to "the offensive capability is published, ship the defense." We can't — the *confidentiality* analog (compilation-induced weight leak via lowering-introduced channels) is **not** in the published attack literature as far as I've seen. Building a tool that identifies which lowerings introduce information channels is, in effect, also building an attack research tool — a SAT counterexample from our checker is a recipe for a confidentiality leak. The mitigations I'd propose: (a) coordinate disclosure with HEIR maintainers and upstream MLIR before publication, in line with standard responsible-disclosure norms for compiler vulnerabilities; (b) prefer to publish the *defensive* result (the data-independent-subset linter and its soundness theorem) early, and the offensive counterexamples only with mitigations in hand; (c) the specific corpus we evaluate against is HEIR (research-stage, narrowly deployed) rather than production XLA or TVM, which limits near-term blast radius. None of this fully resolves the hazard. It is the single most important thing I'd want Atlas's input on before the project goes public.

R6. The fellow is wrong for the project. This is a heavy-prereq project. Required: comfortable in C++17/20 (mlir-tv is C++), comfortable reading MLIR / TableGen, undergraduate-level familiarity with SMT (Z3 / CVC5 invocation, not solver internals), and ideally some exposure to Lean 4 for the stretch. The fellow without the C++ / MLIR comfort will spend most of the fellowship onboarding and produce nothing. **Mitigation:** clear prereq filter in the fellowship application; offer a 1-week "trial PR" before formal acceptance where the candidate adds a simple pattern to the existing verifier — if that takes >5 days, the match isn't right.

R7. The mechanized stretch goal (OP-6, Lean) is a side quest that eats the main goal. Lean mechanization is satisfying and produces clean theorems but doesn't directly serve the

SMT validator. If the fellow drifts toward Lean prematurely, the validator never ships. **Mitigation:** explicit milestone gating (see timeline). Lean work does not begin until the SMT validator hits its M3 milestone.

Capacity

I can commit **~7 hours per week** as primary mentor, broken roughly as: 1 hour weekly sync with the fellow, 2 hours of async PR review / code-review / paper-draft review, 1 hour of literature / venue strategy as deadlines approach, ~2 hours of unblocking / pairing as needed. Higher (8–15 hours) for the two crunch weeks immediately preceding the chosen venue's submission deadline.

I will not commit beyond what I can actually deliver. If real life intervenes (it sometimes does), I'd flag it to the fellowship coordinator within the same week rather than letting the fellow drift.

I would want a co-mentor from Atlas (or another formal-methods academic) for the Lean-side review if the stretch goal activates, since my Lean is intermediate at best. Atlas's existing connections with the [lean-mlir](#) maintainers (opencompl group) would be a force multiplier here.

Proposed timeline / milestones

Fellowship assumed: ~12 weeks, full-time fellow.

None

Milestone:	M1 – Foundation
Description:	Reproducible build of the existing prototype on a clean Linux box. Systematic literature review of non-interference + translation validation + MLIR-adjacent IFC work, with a written ~5-page state-of-the-art summary. Fellow demonstrates fluency by adding two new verifier tests and fixing any bugs surfaced.
Target completion:	Week 3
Key deliverables:	- Buildable artifact in fellow's environment - Lit-review document committed to repo - 2 new tests + any fixes merged - Go / no-go decision on the "first to combine TV with NI in MLIR" framing, based on lit review

None

Milestone:

M2 – Relational SMT encoding (prototype)

Description:

Self-composition transform implemented as an MLIR pass. mlir-tv extended to (a) accept HEIR `secret` dialect ops, (b) accept self-composed programs, (c) treat `aisec.protected` and `aisec.declassify` as first-class. End-to-end pipeline: take a HEIR MLIR program, run lowering pass P, check non-interference preservation via Z3.

Target completion:

Week 7

Key deliverables:

- Code: self-composition pass + mlir-tv extension
- Working pipeline on 3 hand-picked lowerings
- First counterexample (real or constructed) demonstrating a non-interference-violating lowering caught by the tool
- Go / no-go on full evaluation: if encoding doesn't scale to even small inputs, replan

None

Milestone:

M3 – Evaluation + paper draft

Description:

Run the tool on the full ~10-lowering corpus. Quantify: how many lowerings preserve NI by construction (data-independent subset), how many don't, of those how many produce counterexamples the solver can find within budget. Implement the data-independent subset linter pass as a separate deliverable. Write the paper.

Target completion:

Week 11

Key deliverables:

- Full evaluation table + counterexample gallery
- Linter pass + 100% precision/recall measurement on hand-labeled test corpus
- Paper draft (≥80% complete) ready for mentor and co-author review
- Decision on submission venue based on what the evidence will support (CSF preferred; TACAS or FMCAD fallback)

None

Milestone:

M4 (stretch) – Lean mechanization (OP-6)

Description:

If M3 lands cleanly and there is calendar slack before the venue deadline: mechanize the soundness theorem of the data-independent subset linter in lean-mlir. Define the `secret` dialect's four-op subset in Lean. Prove that the data-independent rewrite class preserves non-interference, as a

Lean theorem, no sorry. The lean-mlir vehicle and proof discipline is established by Bhat et al., Verifying Peephole Rewriting in SSA Compiler IRs, ITP 2024 (arxiv.org/abs/2407.03685; repo: github.com/opencompl/lean-mlir).

Target completion: Week 12 + buffer to deadline

Key deliverables:

- Lean source upstreamed (or PR-ready) to opencompl/lean-mlir
- Theorem statement and proof
- Paper revised to include the mechanization as a separate section

Gating: **M4 does not start unless M3 is on track.** The fellow will have a written "stop M4 if" condition at start.

Anything else you'd like us to know

Three things, kept short.

First, this repo's exobrain ([docs/exobrain/](#)) is the research trail for the prototype: critique of the original framework, finding-by-finding citation work, the open-problems triage that produced the seven candidate research questions, and the honest write-up of the toolchain failure that became OP-7. It's there to demonstrate that the project has been red-teamed against itself before being proposed externally. A fellowship reviewer who wants to assess whether the project is grounded should read [docs/exobrain/critique.agents.md](#) and [docs/exobrain/open-problems.agents.md](#).

Second, the closest competitor work I'm aware of — Regehr et al.'s [First-Class Verification Dialects for MLIR](#) (PLDI 2025) — is infrastructure we *use*, not work we displace. We are not racing Regehr's group; if anything, a successful project here is positioned to be a "real-world application" citation for their framework. I'd welcome a Regehr-group co-author on the final paper if they're game.

Third, the dual-use posture (R5 above) is something I'd want to discuss with Atlas before the project goes public. The dominant view in the IFC research community is that publication of non-interference analysis tools is net-defensive (this is the SecWasm / FaCT / Jasmin posture), and I share that view, but the ML-compiler-attack literature is recent enough that the community norms aren't fully settled. Atlas likely has more context here than I do.